INPUT-OUTPUT ORGANIZATION

Mr. SUBHASIS DASH
SCHOLE OF COMPUTER ENGINEERING.
KIIT UNIVERSITY, BHUBANESWAR , ORISSA

INPUT-OUTPUT ORGANIZATION

- Peripheral Devices
- Input-Output Interface
- Asynchronous Data Transfer
- Modes of Transfer
- Priority Interrupt
- Direct Memory Access
- Input-Output Processor
- Serial Communication

PERIPHERAL DEVICES

Input Devices

- Keyboard
- Optical input devices
  - Card Reader
  - Paper Tape Reader
  - Bar code reader
  - Digitizer
  - Optical Mark Reader
- Magnetic Input Devices
  - Magnetic Stripe Reader
- Screen Input Devices
  - Touch Screen
  - Light Pen
  - Mouse
- Analog Input Devices

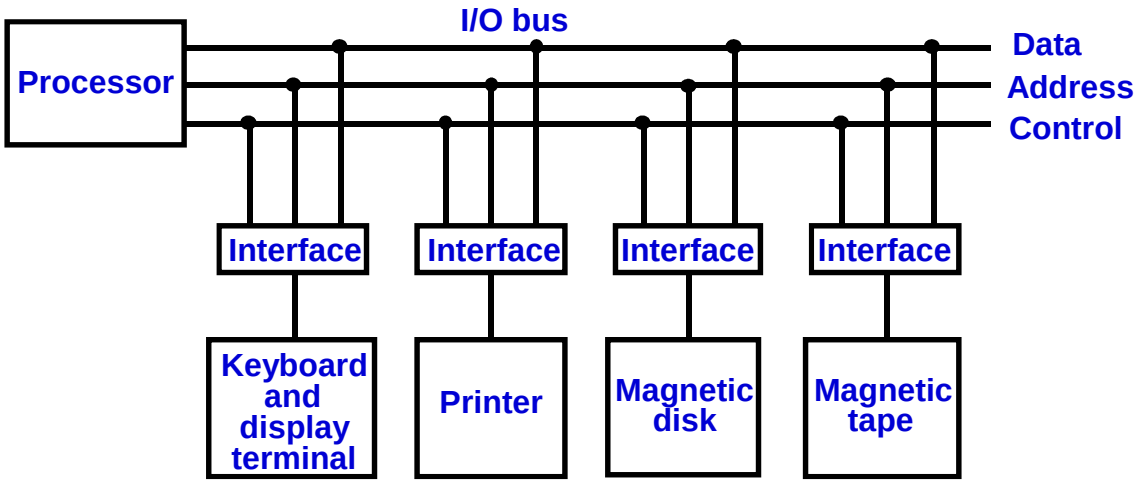
Output Devices

- Card Puncher, Paper Tape Puncher
- CRT
- Printer (Impact, Ink Jet, Laser, Dot Matrix)
- Plotter
- Analog
- Voice

INPUT/OUTPUT INTERFACE

- **Provides a method for transferring information between internal storage (such as memory and CPU registers) and external I/O devices**
- **Resolves the *differences* between the computer and peripheral devices**
  - **Peripherals - Electromechanical Devices**
  - **CPU or Memory - Electronic Device**
  - **Data Transfer Rate**
    - » **Peripherals - Usually slower**
    - » **CPU or Memory - Usually faster than peripherals**
      - **Some kinds of Synchronization mechanism may be needed**
  - **Unit of Information**
    - » **Peripherals – Byte, Block, …**
    - » **CPU or Memory – Word**

I/O BUS AND INTERFACE MODULES



Each peripheral has an interface module associated with it

Interface

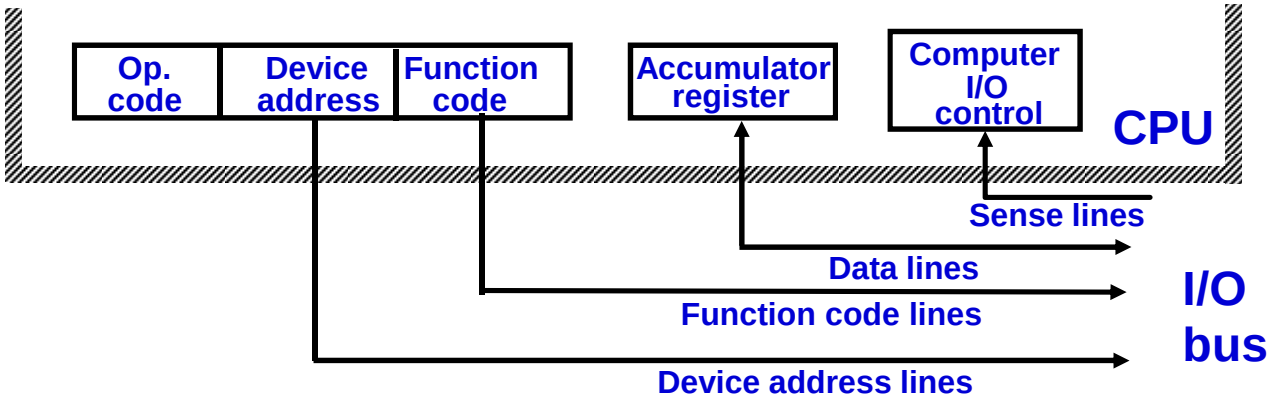
- **Decodes the device address (device code)**
- **Decodes the commands (operation)**
- **Provides signals for the peripheral controller**
- **Synchronizes the data flow and supervises the transfer rate between peripheral and CPU or Memory**

Typical I/O instruction

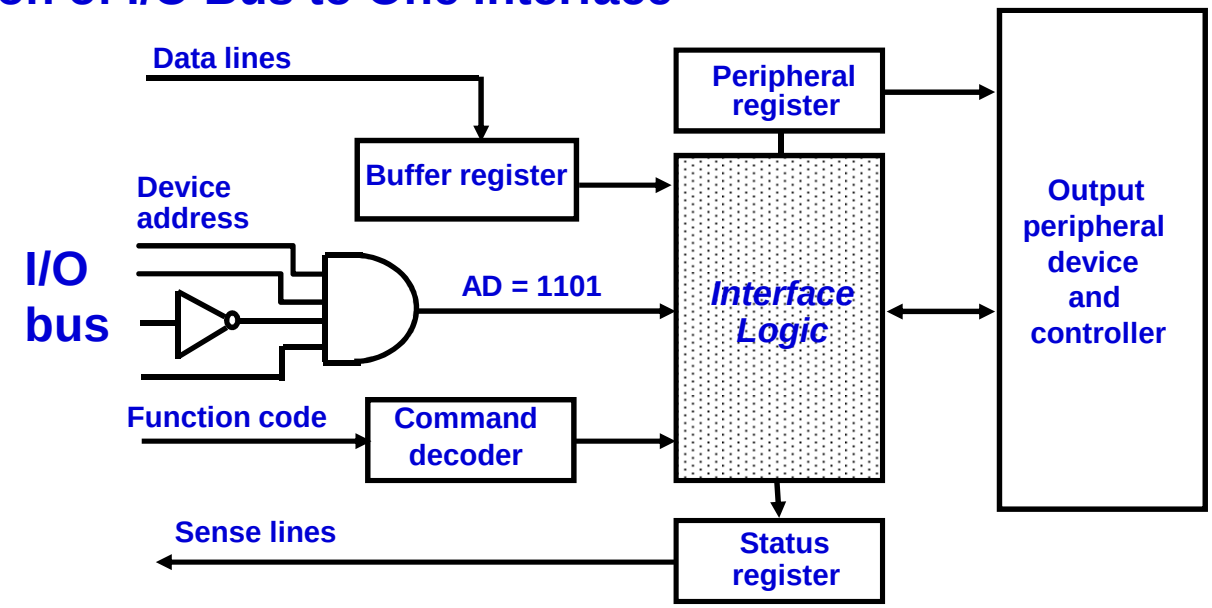
Op. code	Device address	Function code
		(Command)

CONNECTION OF I/O BUS

Connection of I/O Bus to CPU



Connection of I/O Bus to One Interface



I/O BUS AND MEMORY BUS

Functions of Buses

*** *MEMORY BUS* is for information transfers between CPU and the MM**

*** *I/O BUS* is for information transfers between CPU and I/O devices through their I/O interface**

Physical Organizations

*** Many computers use a common single bus system for both memory and I/O interface units**

- **Use one common bus but separate control lines for each function**
- **Use one common bus with common control lines for both functions**

I/O Bus

*** Some computer systems use two separate buses, one to communicate with memory and the other with I/O interfaces**

- **Communication between CPU and all interface units is via a common I/O Bus**
- **An interface connected to a peripheral device may have a number of *data registers* , a *control register*, and a *status register***
- **A command is passed to the peripheral by sending to the appropriate interface register**
- **Function code and sense lines are not needed (Transfer of data, control, and status information is always via the common I/O Bus)**

ISOLATED vs MEMORY MAPPED I/O

Isolated I/O

- Separate I/O read/write control lines in addition to memory read/write control lines
- Separate (isolated) memory and I/O address spaces
- Distinct input and output instructions

Memory-mapped I/O

- A single set of read/write control lines

  (no distinction between memory and I/O transfer)

- Memory and I/O addresses share the common address space

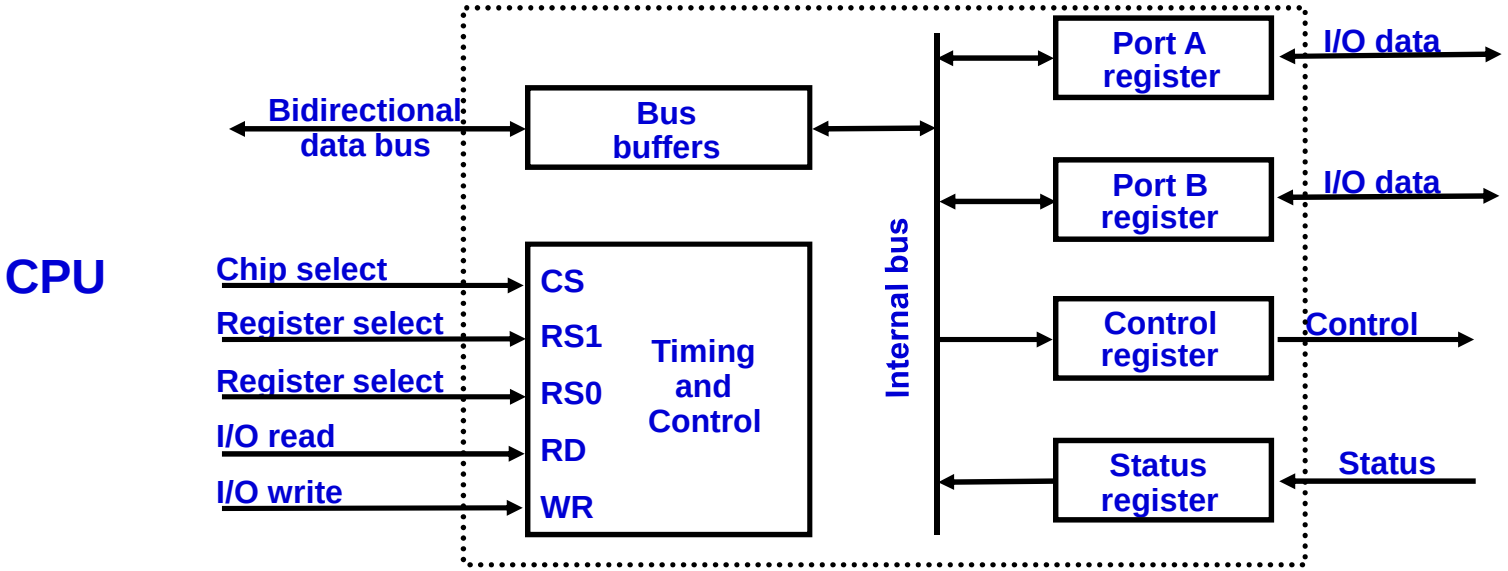
  -> reduces memory address range available

- No specific input or output instruction

  -> The same memory reference instructions can be used for I/O transfers

- Considerable flexibility in handling I/O operations

I/O INTERFACE



CS	RS1	RS0	Register selected
0	x	x	None - data bus in high-impedence
1	0	0	Port A register
1	0	1	Port B register
1	1	0	Control register
1	1	1	Status register

Programmable Interface

- Information in each port can be assigned a meaning depending on the mode of operation of the I/O device
  - → Port A = Data; Port B = Command; Port C = Status
- CPU initializes(loads) each port by transferring a byte to the Control Register
  - → Allows CPU can define the mode of operation of each port
  - → *Programmable Port*: By changing the bits in the control register, it is possible to change the interface characteristics

ASYNCHRONOUS DATA TRANSFER

Synchronous and Asynchronous Operations

Synchronous - All devices derive the timing information from common clock line

Asynchronous - No common clock

Asynchronous Data Transfer

Asynchronous data transfer between two independent units requires that *control signals* be transmitted between the communicating units *to indicate the time at which data is being transmitted*

Two Asynchronous Data Transfer Methods

Strobe pulse

- A strobe pulse is supplied by one unit to indicate the other unit when the transfer has to occur

Handshaking

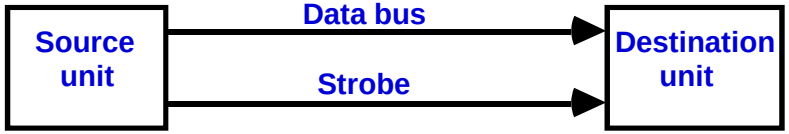
- A control signal is accompanied with each data being transmitted to indicate the presence of data
- The receiving unit responds with another control signal to acknowledge receipt of the data

STROBE CONTROL

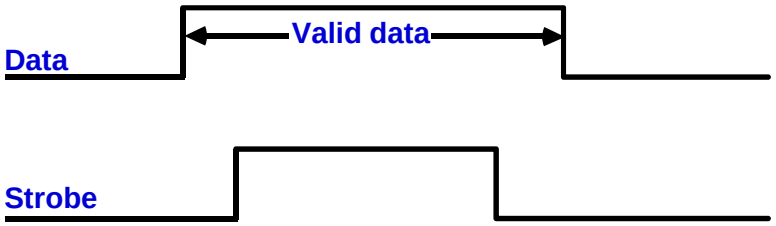
* Employs a single control line to time each transfer
* The strobe may be activated by either the source or the destination unit

Source-Initiated Strobe for Data Transfer

Block Diagram



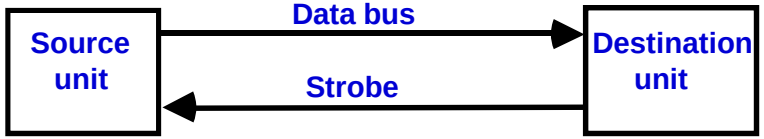
Timing Diagram



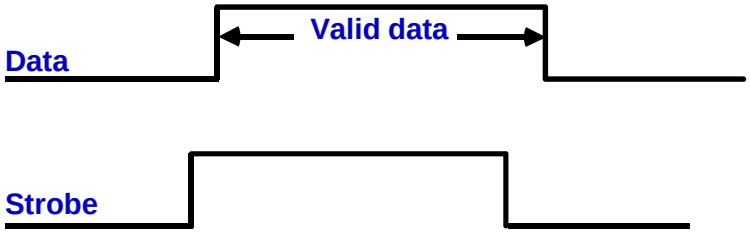
The source unit that initiates the transfer has no way of knowing whether the destination unit has actually received data

Destination-Initiated Strobe for Data Transfer

Block Diagram



Timing Diagram



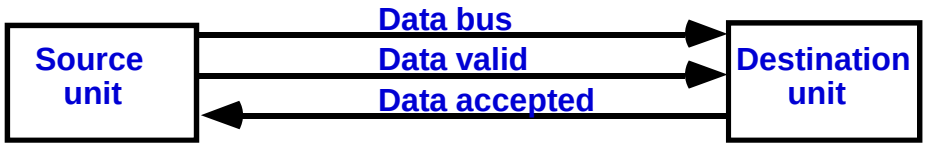
The destination unit that initiates the transfer no way of knowing whether the source has actually placed the data on the bus

HANDSHAKING

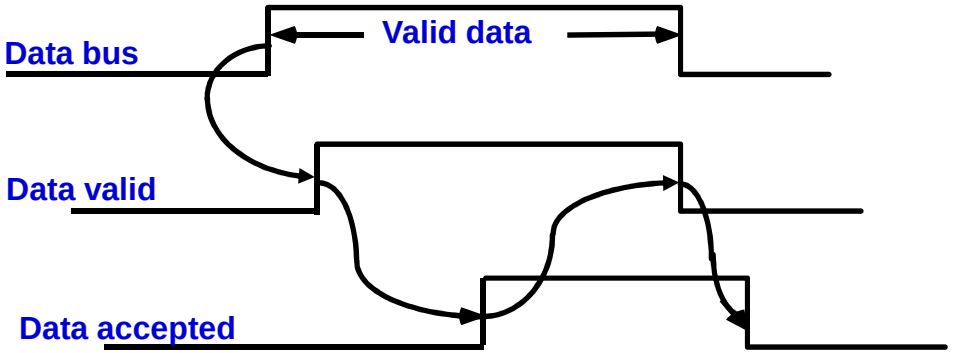
To solve this problem, the *HANDSHAKE* method introduces a second control signal to provide a *Reply* to the unit that initiates the transfer

SOURCE-INITIATED TRANSFER USING HANDSHAKE

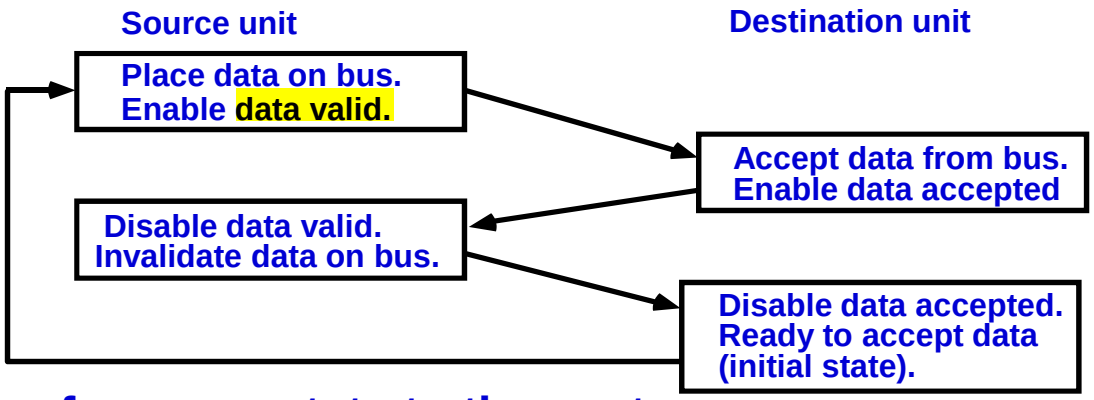
Block Diagram



Timing Diagram



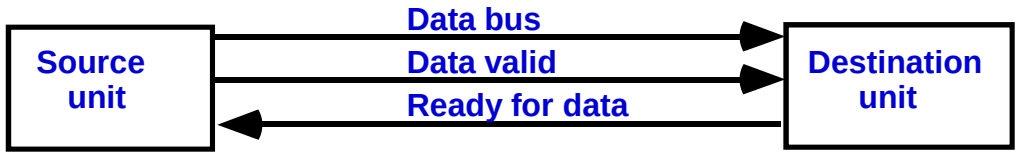
Sequence of Events



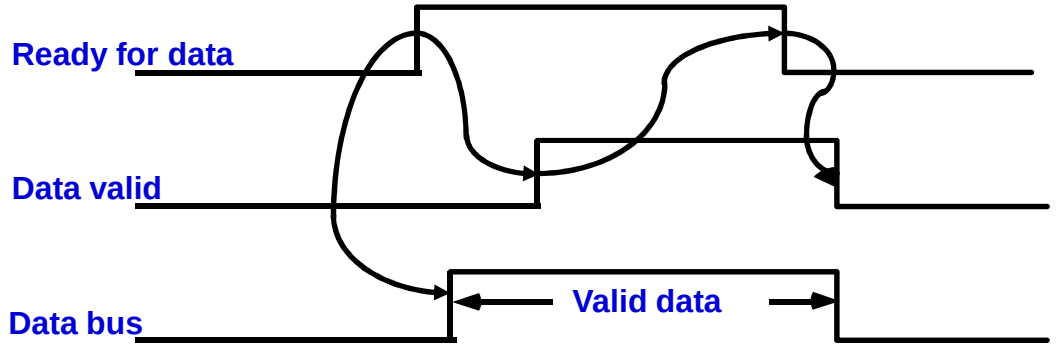
* Allows arbitrary delays from one state to the next
* Permits each unit to respond at its own data transfer rate
* The rate of transfer is determined by the slower unit

DESTINATION-INITIATED TRANSFER USING HANDSHAKE

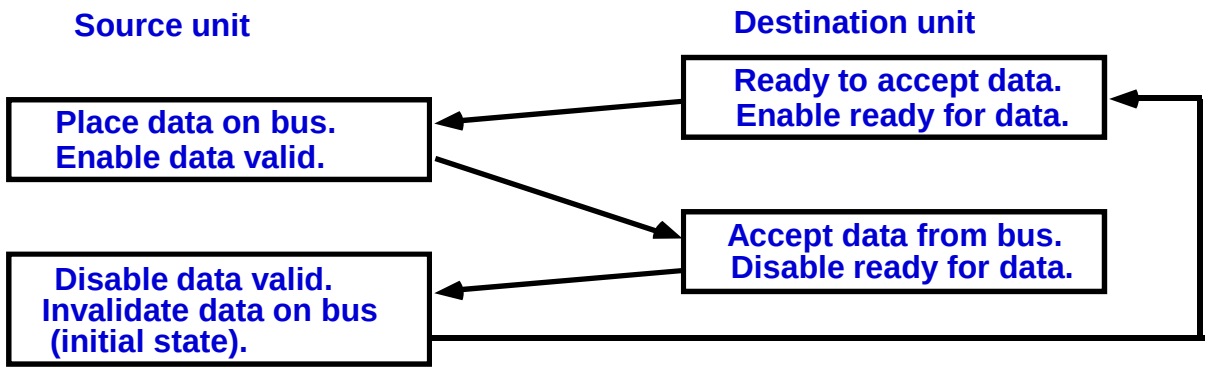
Block Diagram



Timing Diagram



Sequence of Events



* Handshaking provides a high degree of flexibility and reliability because the successful completion of a data transfer relies on active participation by both units
* If one unit is faulty, data transfer will not be completed
  -> Can be detected by means of a *timeout* mechanism

MODES OF TRANSFER

3 different Data Transfer Modes between the central computer(CPU or Memory) and peripherals;

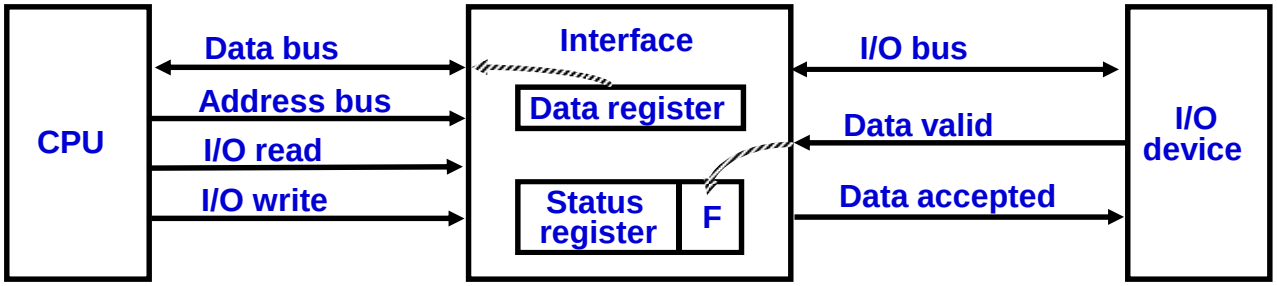
1. **Program-Controlled I/O**
2. **Interrupt-Initiated I/O**
3. **Direct Memory Access (DMA)**

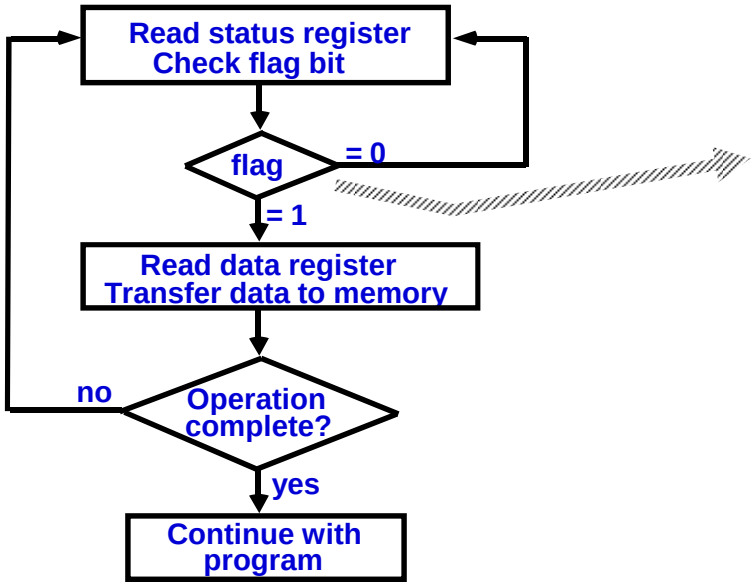
Program-Controlled I/O(Input Dev to CPU

- **In program-controlled I/O, when the processor continuously monitors the status of the device, it does not perform any useful tasks.**
- **An alternate approach would be for the I/O device to alert the processor when it becomes ready.**
  - **Do so by sending a hardware signal called an interrupt to the processor.**
  - **At least one of the bus control lines, called an interrupt-request line is dedicated for this purpose.**
- **Processor can perform other useful tasks while it is waiting for the device to be ready.**

Program-Controlled I/O(Input Dev to CPU

1. Read the status register.
2. Check the status of the flag bit and branch to step 1 if not set or to step 3 if set.
3. Read the data register.





Polling or Status Checking

- Continuous CPU involvement
- CPU slowed down to I/O speed
- Simple
- Least hardware

Interrupt Initiated I/O

1. **Polling takes valuable CPU time**

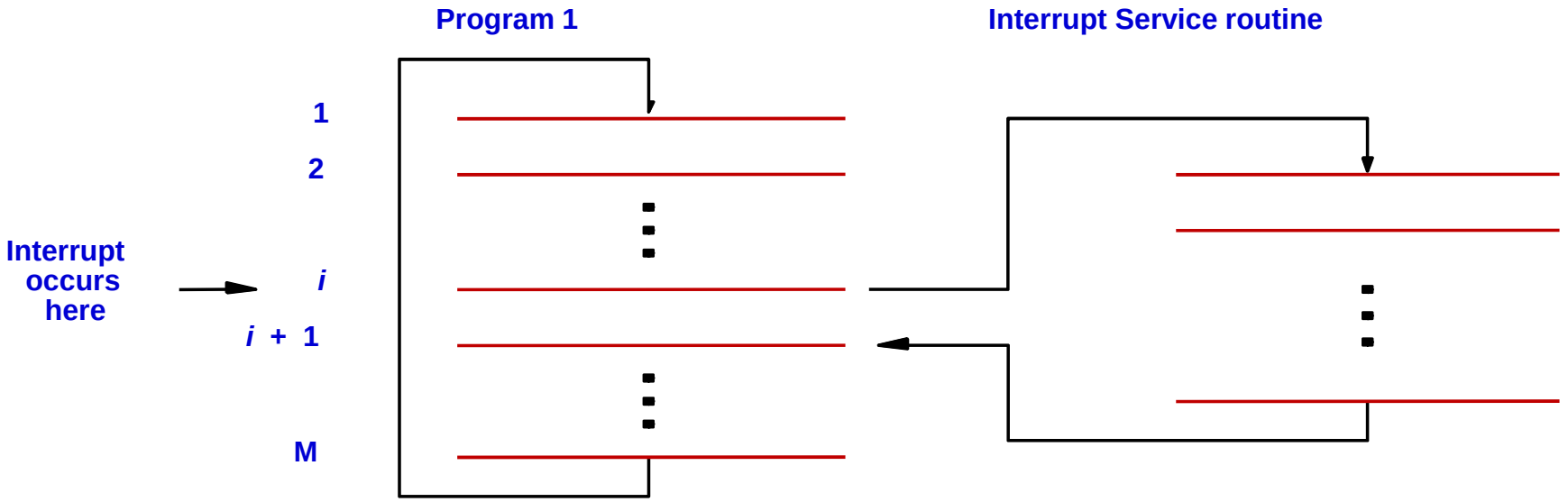
2. **Open communication only when some data has to be passed ➔*Interrupt*.**

3. **I/O interface, instead of the CPU, monitors the I/O device**

4. **When the interface determines that the I/O device is ready for data transfer, it generates an *Interrupt Request* to the CPU**

5. **Upon detecting an interrupt, CPU stops momentarily the task it is doing, branches to the service routine to process the data transfer, and then returns to the task it was performing**

Interrupt Initiated I/O



1. *Processor is executing instruction located at address i when an interrupt occurs.*
2. *Routine executed in response to an interrupt request is called the interrupt-service routine.*
3. *When an interrupt occurs, control must be transferred to the interrupt service routine.*
4. *But before transferring control, the current contents of the PC (i+1), must be saved in a known location.*
5. *This will enable the return-from-interrupt instruction to resume execution at i+1.*
6. *Return address, or the contents of the PC are usually stored on the processor stack.*

Interrupt Initiated I/O

Interrupt Latency

Saving and restoring information can be done automatically by the processor or explicitly by program instructions.

Saving and restoring registers involves memory transfers:

- **Increases the total execution time.**
- **Increases the delay between the time an interrupt request is received, and the start of execution of the interrupt-service routine. This delay is called interrupt latency.**

In order to reduce the interrupt latency, most processors save only the minimal amount of information:

- **This minimal amount of information includes Program Counter and processor status registers.**

Any additional information that must be saved, must be saved explicitly by the program instructions at the beginning of the interrupt service routine.

Interrupt Initiated I/O

Enable & Disable

- When a processor receives an interrupt-request, it must branch to the interrupt service routine.
- It must also inform the device that it has recognized the interrupt request.
- This can be accomplished in two ways:
  - Some processors have an explicit interrupt-acknowledge control signal for this purpose.
  - In other cases, the data transfer that takes place between the device and the processor can be used to inform the device.
- Processors generally provide the ability to enable and disable interruptions as desired.
- To avoid interruption by the same device during the execution of an interrupt service routine:
  - ✓ First instruction of an interrupt service routine can be Interrupt-disable.
  - ✓ Last instruction of an interrupt service routine can be Interrupt-enable.

Interrupt Initiated I/O

Assuming that interrupts are enabled in both the processor and the device, the following is a typical scenario➔

1. **The device raises an interrupt request.**
2. **The processor interrupts the program currently being executed and saves the contents of the PC and PS registers.**
3. **Interrupts are disabled by clearing the IE bit in the PS to 0.**
4. **The action requested by the interrupt is performed by the interrupt-service routine, during which time the device is informed that its request has been recognized, and in response, it deactivates the interrupt-request signal.**
5. **Upon completion of the interrupt-service routine, the saved contents of the PC and PS registers are restored (enabling interrupts by setting the IE bit to 1), and execution of the interrupted program is resumed.**

Interrupt Initiated I/O

Handling Multiple Device

Multiple I/O devices may be connected to the processor and the memory via a bus. Some or all of these devices may be capable of generating interrupt requests.

- **Each device operates independently, and hence no definite order can be imposed on how the devices generate interrupt requests?**

How does the processor know which device has generated an interrupt?

How does the processor know which interrupt service routine needs to be executed?

When the processor is executing an interrupt service routine for one device, can other device interrupt the processor?

If two interrupt-requests are received simultaneously, then how to break the tie?

Interrupt Initiated I/O

Polling Mechanism

Consider a simple arrangement where all devices send their interrupt-requests over a single control line in the bus.

When the processor receives an interrupt request over this control line, how does it know which device is requesting an interrupt?

This information is available in the status register of the device requesting an interrupt:

- **The status register of each device has an IRQ bit which it sets to 1 when it requests an interrupt.**

Interrupt service routine can poll the I/O devices connected to the bus. The first device with IRQ equal to 1 is the one that is serviced.

Polling mechanism is easy, but time consuming to query the status bits of all the I/O devices connected to the bus.

Interrupt Initiated I/O

Vectored Mechanism

- **To reduce the time involved in the polling process➔**
- **The device requesting an interrupt may identify itself directly to the processor.**
  - **Device can do so by sending a special code (4 to 8 bits) the processor over the bus.**
  - **Code supplied by the device may represent a part of the starting address of the interrupt-service routine.**
  - **The remainder of the starting address is obtained by the processor based on other information such as the range of memory addresses where interrupt service routines are located.**
- **Usually the location pointed to by the interrupting device is used to store the starting address of the interrupt-service routine.**

Interrupt Initiated I/O

Interrupt Multiple Priority *Mechanism*

- **I/O devices are organized in a priority structure:**
  - **An interrupt request from a high-priority device is accepted while the processor is executing the interrupt service routine of a low priority device.**
- **A priority level is assigned to a processor that can be changed under program control.**
  - **Priority level of a processor is the priority of the program that is currently being executed.**
  - **When the processor starts executing the interrupt service routine of a device, its priority is raised to that of the device.**
  - **If the device sending an interrupt request has a higher priority than the processor, the processor accepts the interrupt request.**

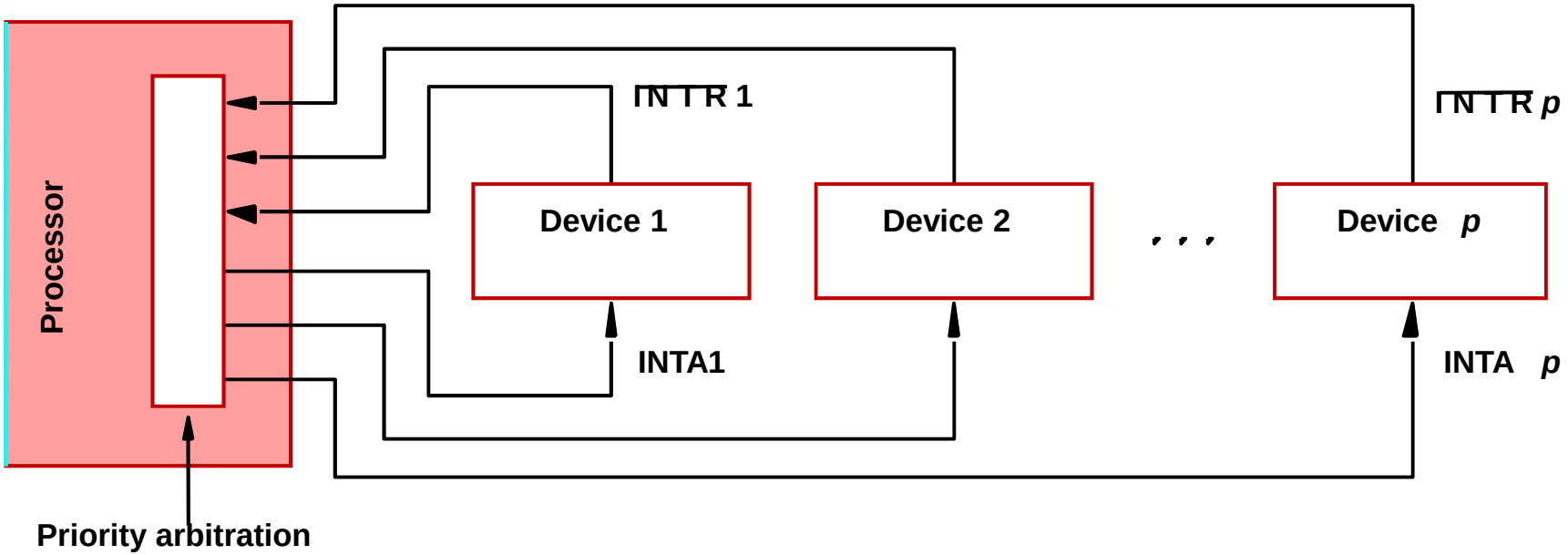
Interrupt Initiated I/O

Interrupt Multiple Priority *Mechanism*

- **Processor's priority is encoded in a few bits of the processor status register.**
  - **Priority can be changed by instructions that write into the processor status register.**
  - **Usually, these are privileged instructions, or instructions that can be executed only in the supervisor mode.**
  - **Privileged instructions cannot be executed in the user mode.**
  - **Prevents a user program from accidentally or intentionally changing the priority of the processor.**
- **If there is an attempt to execute a privileged instruction in the user mode, it causes a special type of interrupt called as privilege exception.**

Interrupt Initiated I/O

Interrupt Multiple Priority *Mechanism*



Implementation of Interrupt Priority using Individual interrupt request and acknowledge lines

1. Each device has a separate interrupt-request and interrupt-acknowledge line.
2. Each interrupt-request line is assigned a different priority level.
3. Interrupt requests received over these lines are sent to a priority arbitration circuit in the processor.
4. If the interrupt request has a higher priority level than the priority of the processor, then the request is accepted.

Interrupt Initiated I/O

Simultaneous Interrupt *Request*

Which interrupt request does the processor accept if it receives interrupt requests from two or more devices simultaneously?

If the I/O devices are organized in a priority structure, the processor accepts the interrupt request from a device with higher priority.

- **Each device has its own interrupt request and interrupt acknowledge line.**
- **A different priority level is assigned to the interrupt request line of each device.**

However, if the devices share an interrupt request line, then how does the processor decide which interrupt request to accept?

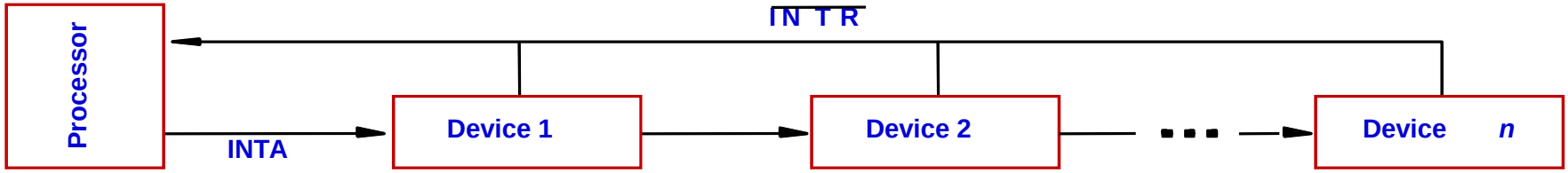
Interrupt Initiated I/O

Simultaneous Interrupt *Request*

Polling scheme➔

1. If the processor uses a polling mechanism to poll the status registers of I/O devices to determine which device is requesting an interrupt.
2. In this case the priority is determined by the order in which the devices are polled.
3. The first device with status bit set to 1 is the device whose interrupt request is accepted.

Daisy chain scheme➔



1. Devices are connected to form a daisy chain.
2. Devices share the interrupt-request line, and interrupt-acknowledge line is connected to form a daisy chain.
3. When devices raise an interrupt request, the interrupt-request line is activated.
4. The processor in response activates interrupt-acknowledge.
5. Received by device 1, if device 1 does not need service, it passes the signal to device 2.
6. Device that is electrically closest to the processor has the highest priority.

Direct Memory Access

Direct Memory Access (DMA):

- **A special control unit may be provided to transfer a block of data directly between an I/O device and the main memory, without continuous intervention by the processor.**

Control unit which performs these transfers is a part of the I/O device's interface circuit. This control unit is called as a DMA controller.

DMA controller performs functions that would be normally carried out by the processor:

- **For each word, it provides the memory address and all the control signals.**
- **To transfer a block of data, it increments the memory addresses and keeps track of the number of transfers.**

Direct Memory Access

DMA controller can transfer a block of data from an external device to the processor, without any intervention from the processor.

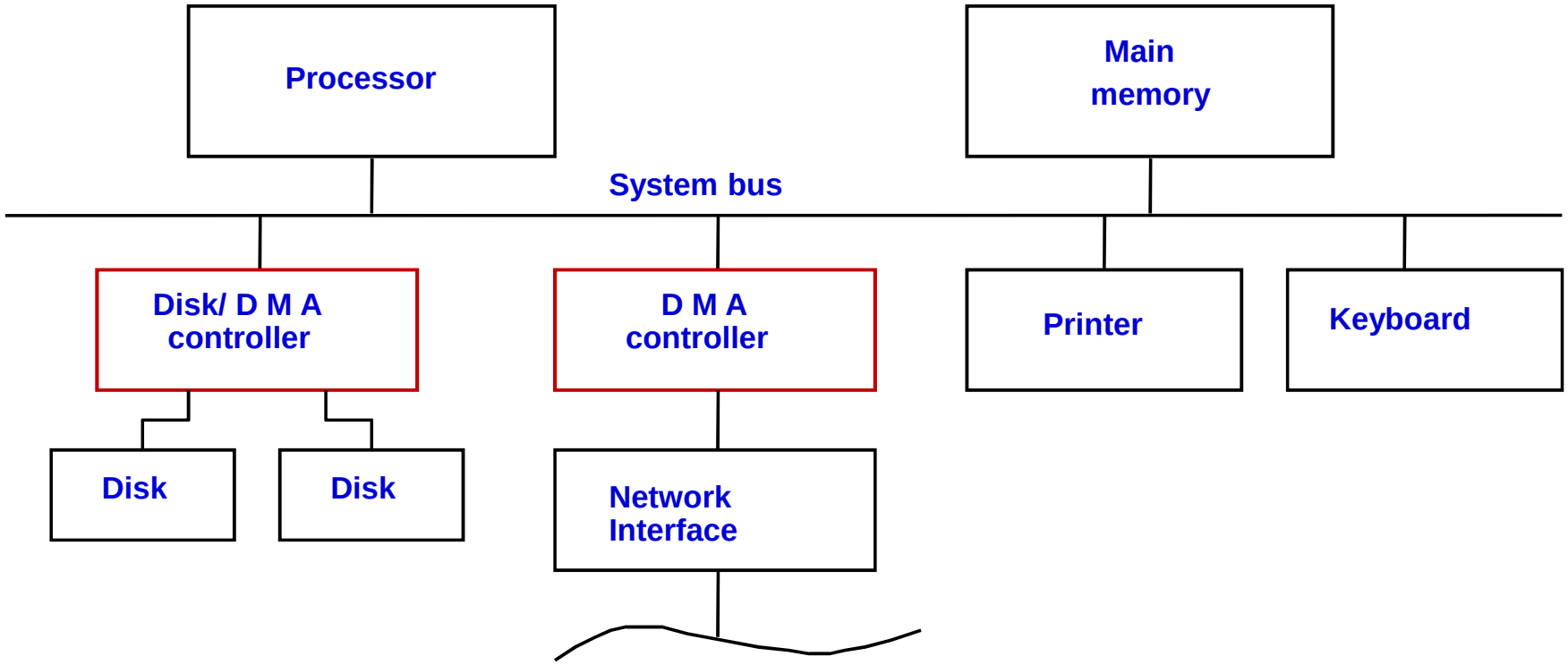
- **However, the operation of the DMA controller must be under the control of a program executed by the processor. That is, the processor must initiate the DMA transfer.**

To initiate the DMA transfer, the processor informs the DMA controller of:

- **Starting address,**
- **Number of words in the block.**
- **Direction of transfer (I/O device to the memory, or memory to the I/O device).**

Once the DMA controller completes the DMA transfer, it informs the processor by raising an interrupt signal.

Direct Memory Access



1. DMA controller connects a high-speed network to the computer bus.
2. Disk controller, which controls two disks also has DMA capability. It provides two DMA channels.
3. It can perform two independent DMA operations, as if each disk has its own DMA controller. The registers to store the memory address, word count and status and control information are duplicated.

Direct Memory Access

Processor and DMA controllers have to use the bus in an interwoven fashion to access the memory.

- **DMA devices are given higher priority than the processor to access the bus.**
- **Among different DMA devices, high priority is given to high-speed peripherals such as a disk or a graphics display device.**

Processor originates most memory access cycles on the bus.

- **DMA controller can be said to "steal" memory access cycles from the bus. This interweaving technique is called as "cycle stealing".**

An alternate approach is the provide a DMA controller an exclusive capability to initiate transfers on the bus, and hence exclusive access to the main memory. This is known as the block or burst mode.

Direct Memory Access

Bus arbitration

Processor and DMA controllers both need to initiate data transfers on the bus and access main memory.

The device that is allowed to initiate transfers on the bus at any given time is called the bus master.

When the current bus master relinquishes its status as the bus master, another device can acquire this status.

- **The process by which the next device to become the bus master is selected and bus mastership is transferred to it is called bus arbitration.**

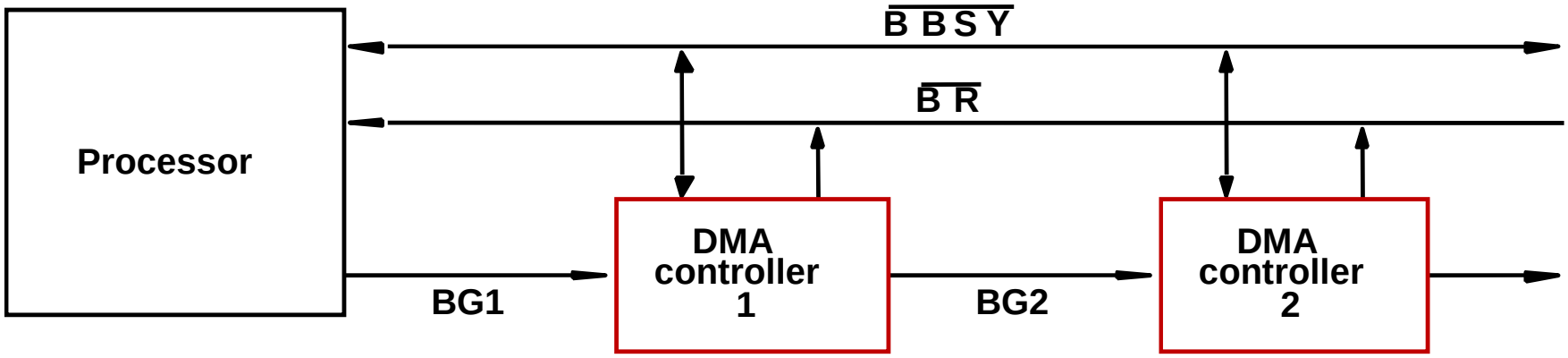
Centralized arbitration:

- **A single bus arbiter performs the arbitration.**

Distributed arbitration:

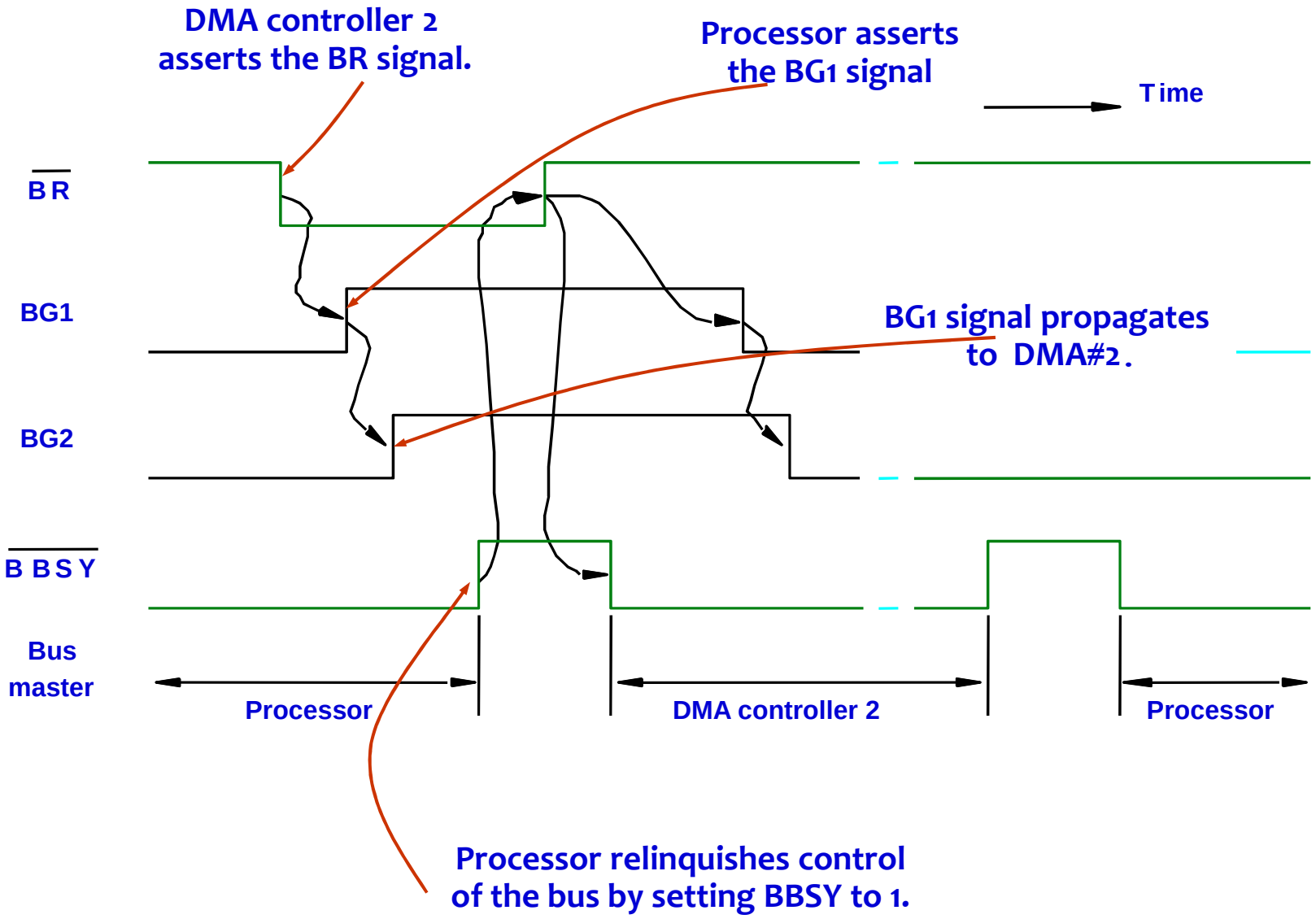
- **All devices participate in the selection of the next bus master.**

Centralized Bus Arbitration



1. Bus arbiter may be the processor or a separate unit connected to the bus.
2. Normally, the processor is the bus master, unless it grants bus membership to one of the DMA controllers.
3. DMA controller requests the control of the bus by asserting the Bus Request (BR) line.
4. In response, the processor activates the Bus-Grant1 (BG1) line, indicating that the controller may use the bus when it is free.
5. BG1 signal is connected to all DMA controllers in a daisy chain fashion.
6. BBSY signal is 0, it indicates that the bus is busy. When BBSY becomes 1, the DMA controller which asserted BR can acquire control of the bus.

Centralized arbitration



Distributed arbitration

All devices waiting to use the bus share the responsibility of carrying out the arbitration process.

- **Arbitration process does not depend on a central arbiter and hence distributed arbitration has higher reliability.**

Each device is assigned a 4-bit ID number.

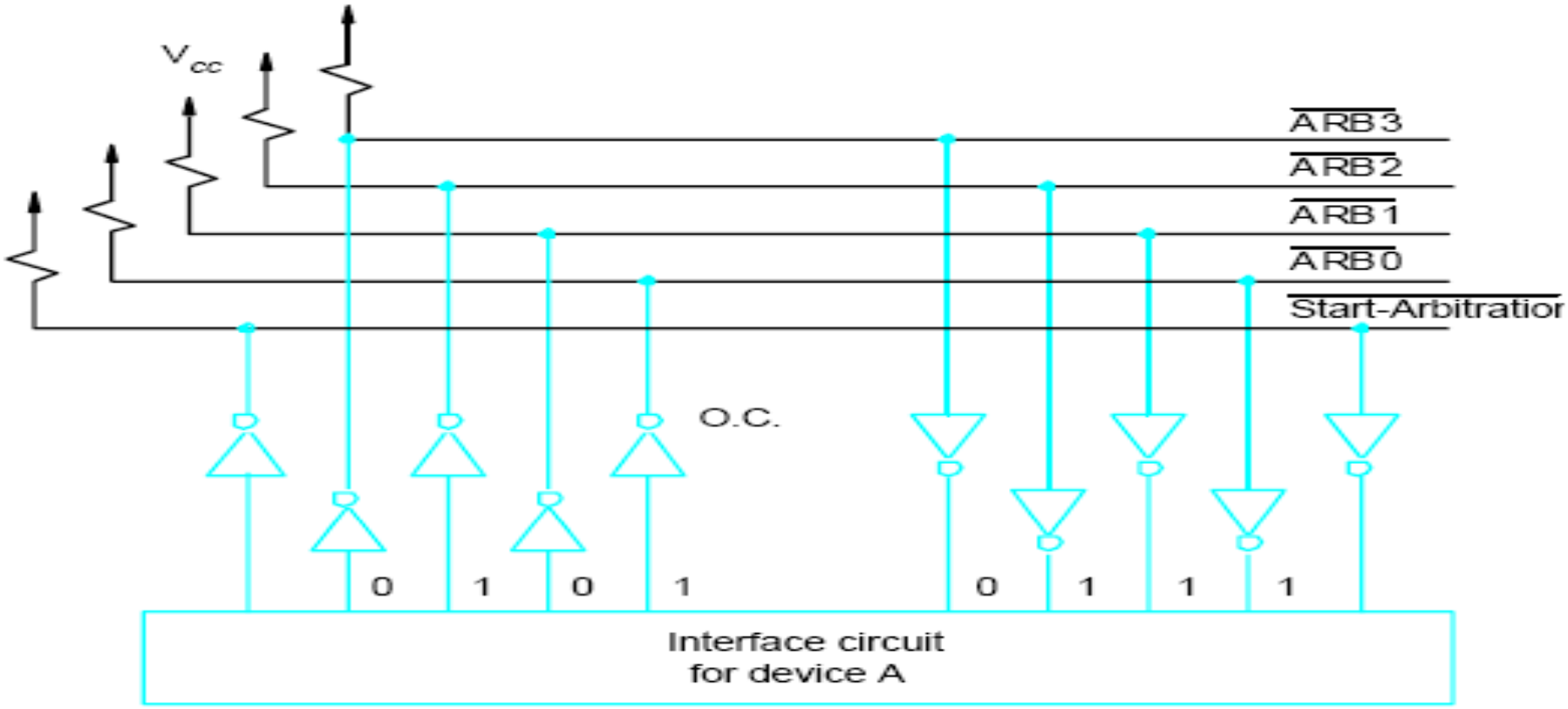
All the devices are connected using 5 lines, 4 arbitration lines to transmit the ID, and one line for the Start-Arbitration signal.

To request the bus a device:

- **Asserts the Start-Arbitration signal.**
- **Places its 4-bit ID number on the arbitration lines.**

The pattern that appears on the arbitration lines is the logical-OR of all the 4-bit device IDs placed on the arbitration lines.

Distributed arbitration



- **Arbitration process:**
  - **Each device compares the pattern that appears on the arbitration lines to its own ID, starting with MSB.**
  - **If it detects a difference, it transmits 0s on the arbitration lines for that and all lower bit positions.**
  - **The pattern that appears on the arbitration lines is the logical-OR of all the 4-bit device IDs placed on the arbitration lines.**

Distributed arbitration

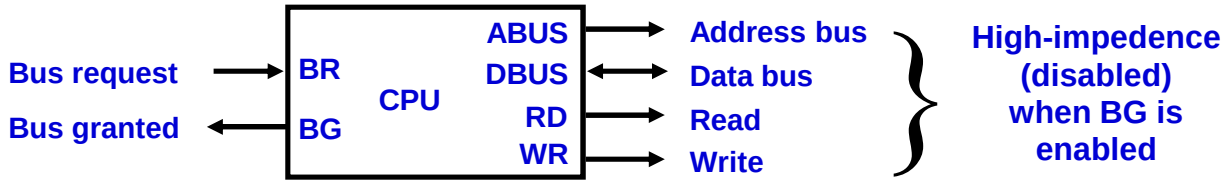
- *Device A has the ID 5 and wants to request the bus:*
  - *Transmits the pattern 0101 on the arbitration lines.*
- *Device B has the ID 6 and wants to request the bus:*
  - *Transmits the pattern 0110 on the arbitration lines.*
- *Pattern that appears on the arbitration lines is the logical OR of the patterns:*
  - *Pattern 0111 appears on the arbitration lines.*

1. **Arbitration process:**
2. **Each device compares the pattern that appears on the arbitration lines to its own ID, starting with MSB.**
3. **If it detects a difference, it transmits 0s on the arbitration lines for that and all lower bit positions.**
4. **Device A compares its ID 5 with a pattern 0101 to pattern 0111.**
5. **It detects a difference at bit position 0, as a result, it transmits a pattern 0100 on the arbitration lines.**
6. **The pattern that appears on the arbitration lines is the logical-OR of 0100 and 0110, which is 0110.**
7. **This pattern is the same as the device ID of B, and hence B has won the arbitration.**

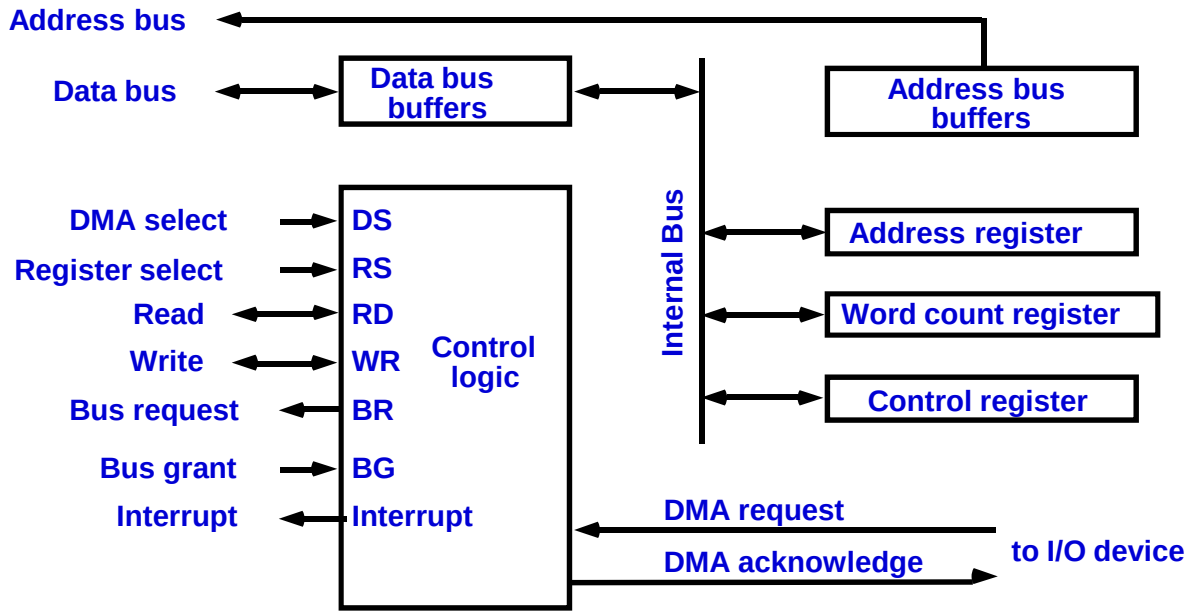
DIRECT MEMORY ACCESS

1. Block of data transfer from high speed devices, Drum, Disk, Tape
2. DMA controller - Interface which allows I/O transfer directly between Memory and Device, freeing CPU for other tasks
3. CPU initializes DMA Controller by sending memory address and the block size(number of words)

CPU bus signals for DMA transfer



Block diagram of DMA controller



DMA TRANSFER